

Assignment 3 (NLP) Writeup

DO NOT TAG

Name: Edward Capone
GT Email: ecapone3@gatech.edu

Seq2Seq Results – Default configuration

Values are for last epoch

RNN

Training Loss: **4.6102**

Training Perplexity: **100.5062**

Validation Loss: **4.7044**

Validation Perplexity: **110.4327**

LSTM

Training Loss: **3.1812**

Training Perplexity: **24.0753**

Validation Loss: **3.3620**

Validation Perplexity: **28.8469**

RNN-with-Attention

Training Loss: **3.3468**

Perplexity: **28.4110**

Validation Loss: **3.4921**

Validation Perplexity: **32.8544**

LSTM-with-Attention

Training Loss: **3.1757**

Training Perplexity: **23.9442**

Validation Loss: **3.3567**

Validation Perplexity: **28.6954**

Seq2Seq Explanation (RNN vs LSTM)

Compare your RNN result to your LSTM result and explain why they differ.

The LSTM model achieves significantly lower perplexities than the vanilla RNN (both with and without attention). This is primarily because LSTMs mitigate the vanishing gradient problem through their gating mechanisms (input, forget, and output gates). These gates allow the network to retain and utilize information over longer sequences, whereas vanilla RNNs tend to lose context as the sequence grows. Consequently, LSTMs capture long-range dependencies more effectively, leading to better overall performance and lower perplexity scores. The benefit becomes even more pronounced when combined with an attention mechanism, as the LSTM's ability to preserve information pairs well with attention's ability to highlight the most relevant parts of the input sequence.

Seq2Seq Explanation (RNN vs RNN-with-Attention)

Compare your RNN result to your RNN-with-Attention result and explain why they differ.

In a standard RNN-based Seq2Seq model, the decoder relies on a single hidden state (the final encoder hidden state) to represent the entire input sequence. This can become a bottleneck, especially for longer sentences, because the model must compress all the relevant source information into one vector. As a result, important details can be lost, and the decoder may struggle to generate accurate translations. On the other hand, adding an attention mechanism allows the decoder to look back at all the encoder's outputs at each decoding step. Instead of depending solely on a final hidden state, the decoder computes "attention weights" to selectively focus on the most relevant parts of the encoder outputs for predicting each target token. This extra flexibility reduces information loss and improves alignment between source and target words. Consequently, the RNN-with-Attention model typically yields lower perplexity scores and more accurate translations, as the decoder can dynamically attend to the context it needs at each step.

Seq2Seq Results – Best model

Values are for last epoch

Your best model after hyper-parameter tuning: **LSTM-with-Attention**

Best model

Training Loss: **2.2488**

Training Perplexity: **9.4763**

Validation Loss: **3.1676**

Validation Perplexity: **23.7499**

List your best model hyper-parameter values including model type: ?

Model Type: LSTM-with-Attention

Learning Rate: 1e-3

Embedding Size: 256

Number of Epochs: 30

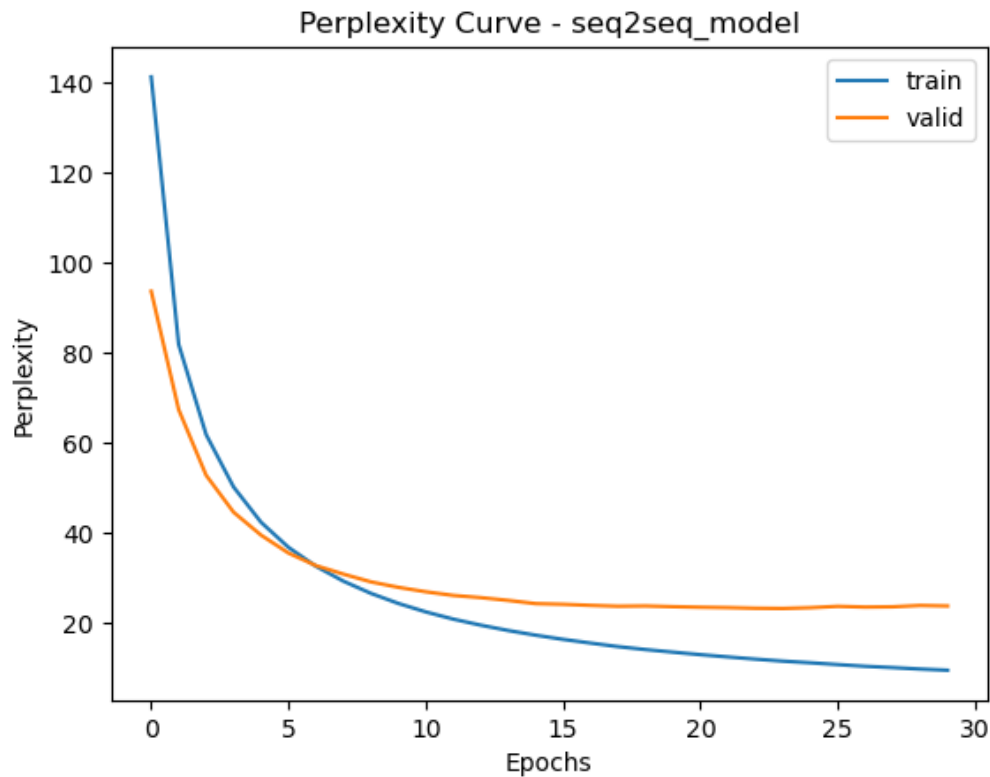
Hidden Size: 256

Optimizer: Adam

Dropout: 0.2

Scheduler: ReduceLROnPlateau

Seq2Seq Best model Learning Curves (Perplexity)



By the end of 30 epochs, our training perplexity reached ~ 9.48 , while the validation perplexity stabilized around ~ 23.75 . The chart shows how training perplexity continued to decrease steadily, whereas validation perplexity plateaued in the mid-20s, indicating our final Seq2Seq model's performance on unseen data.

Seq2Seq Explanation – Best model

Explain the details of your best model. Explain what you did to improve your model's performance and why

Our best Seq2Seq model leverages an LSTM encoder-decoder architecture with attention. We found that introducing attention significantly improved performance by allowing the decoder to focus on the most relevant parts of the encoder's hidden states at each decoding step. After experimenting with various hyperparameter configurations, we settled on an embedding dimension of 256, a hidden size of 256, and a dropout rate of 0.2. We trained for 30 epochs using the Adam optimizer at a learning rate of 1e-3, alongside a learning rate scheduler that reduced the rate when training loss plateaued. These choices balanced model capacity and regularization. The attention mechanism directly addressed the limitation of relying on a single encoder hidden state, particularly for longer sentences. Meanwhile, larger embedding and hidden dimensions gave the model sufficient representational power, and dropout guarded against overfitting. Training longer (30 epochs) also helped the model converge more fully, while monitoring validation perplexity and employing the scheduler prevented overfitting or getting stuck in suboptimal minima. Overall, these strategies led to a training loss of 2.2488 (perplexity 9.4763) and a validation loss of 3.1676 (perplexity 23.7499), a clear improvement over simpler configurations.

Transformer Results

Values are for last epoch

Default configuration (Encoder Only)

Training Loss: **2.1224**
Training Perplexity: **8.3513**
Validation Loss: **3.0687**
Validation Perplexity: **21.5145**

Best model (full transformer)

Training Loss: **0.7839**
Training Perplexity: **2.1901**
Validation Loss: **1.7711**
Validation Perplexity: **5.8771**

Default configuration (full transformer)

Training Loss: **2.0918**
Training Perplexity: **8.0994**
Validation Loss: **2.0536**
Validation Perplexity: **7.7957**

List your best model hyper-parameter values (full transformer): ?

Hidden/Embedding Dimension: 128

Number of Attention Heads: 2

Feedforward Network Dimension: 2048

Learning Rate: 1e-3

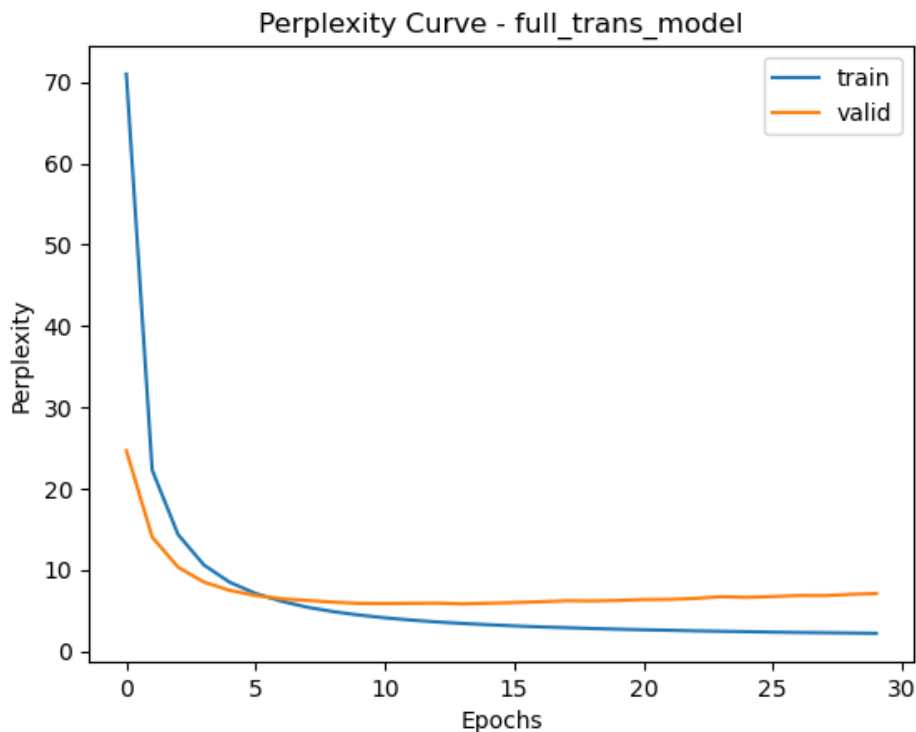
Number of Epochs: 30

Optimizer: Adam

Learning Rate Scheduler: ReduceLROnPlateau

Loss Function: CrossEntropyLoss

Full Transformer Best model Learning Curves (Perplexity)



In this figure, we present the training and validation perplexities for our best full Transformer model over 30 epochs. At the start of training, the model exhibits a high training perplexity (around 70) and a moderately high validation perplexity (approximately 25), reflecting its initial limited understanding of the task. However, both curves decline rapidly as training proceeds. By around epoch 10–15, the training perplexity falls below 10, while the validation perplexity settles around 6–8, indicating effective generalization. The relatively small gap between the two curves suggests that overfitting is not a significant concern. Overall, these learning curves demonstrate the Transformer's ability to capture the underlying patterns of the data and achieve strong performance in terms of perplexity.

Full Transformer Explanation – Best model

Explain what you did here and why you did it to improve your model performance.

In our best full Transformer model, we focused on balancing model complexity with efficient training to achieve strong performance. Specifically, we used a hidden/embedding dimension of 128, two attention heads, and a feedforward dimension of 2048. This setup provides sufficient capacity for the Transformer to learn nuanced relationships in the data while remaining computationally feasible. We trained for 30 epochs with a $1e-3$ learning rate, using the Adam optimizer and a ReduceLROnPlateau scheduler to manage learning rate adjustments when improvements plateaued. By carefully tuning these hyperparameters, we ensured that the model converged smoothly, as evidenced by steadily decreasing perplexities on both the training and validation sets. The relatively small gap between these perplexities further indicates that our chosen configuration mitigates overfitting, allowing the Transformer to generalize effectively and deliver robust performance.

Transformer (Encoder Only) Translation Results (default settings)

Put translation results for your model (1st 9 sentences) here. You may remove duplicate <pad> and <eos> tokens for each sentence.

True Translation	Predicted Translation
a man in an orange hat starring at something	a man in an orange hat hat is something
a boston terrier is running on lush green grass in front of a white fence	a boston terrier runs running grass green grass grass in of white white fence fence fence
a girl in karate uniform breaking a stick with a front kick	a girl in a a a large is with a a
five people wearing winter jackets and helmets stand in the snow with <unk> in the background	five people in in and in helmets are in in snow in in in in
people are fixing the roof of a house	people repairing admiring the of view house
a group of people standing in front of an igloo	a group of people stand stand front front front
a guy works on a building	a guy works working on building
a man in a vest is sitting in a chair and holding magazines	a man in a vest is is sitting a chair a holding
a mother and her young song enjoying a beautiful day outside	a mother and her enjoying son son enjoying enjoying beautiful day

Table 1

Full Transformer Translation Results (best model)

Put translation results for your best model (1 - 9 sentences) here. You may remove duplicate <pad> and <eos> tokens for each sentence.

True Translation	Predicted Translation
a man in an orange hat starring at something	a man in an orange hat is doing something with an orange toothbrush <unk>
a boston terrier is running on lush green grass in front of a white fence	a boston terrier runs across the green grass in front of a white fence <unk>
a girl in karate uniform breaking a stick with a front kick	a girl in a karate uniform is performing a stunt <unk>
five people wearing winter jackets and helmets stand in the snow with <unk> in the background	five people in winter jackets and helmets are standing in the snow with tall buildings in the background <unk>
people are fixing the roof of a house	people are fixing the roof of a house <unk>
a group of people standing in front of an igloo	a group of people are standing in front of an igloo <unk>
a guy works on a building	a guy works on a building <unk>
a man in a vest is sitting in a chair and holding magazines	a man in a vest is sitting on a chair holding a marker <unk>
a mother and her young song enjoying a beautiful day outside	a mother and her son enjoying a nice day outdoors <unk>

Table 2

Seq2Seq (Best model) Translation Results

Put translation results for your best model (1 - 9 sentences) here. You may remove duplicate <pad> and <eos> tokens for each sentence.

True Translation	Predicted Translation
a man in an orange hat starring at something	a man in a orange is welding
a boston terrier is running on lush green grass in front of a white fence	a green green is running through a a field of a a green green
a girl in karate uniform breaking a stick with a front kick	a girl in a karate uniform is a a a a a
five people wearing winter jackets and helmets stand in the snow with <unk> in the background	five people in in in in are are in in in in in in background background background
people are fixing the roof of a house	people are the the the a a of
a group of people standing in front of an igloo	a group of people standing in front of a
a guy works on a building	a guy is working on a building building
a man in a vest is sitting in a chair and holding magazines	a man in a yellow vest sitting sitting a a chair
a mother and her young son enjoying a beautiful day outside	a mother and her daughter her her a a a

Table 3

Compare Transformer (Encoder Only) to Transformer (Full transformer)

Compare your results for default settings for Encoder Only Transformer vs best model for Full transformer both quantitatively and qualitatively. Explain why you see differences.

In comparing the Transformer (Encoder Only) model under default settings to the Full Transformer model at its best settings, we observe clear differences both quantitatively and qualitatively. Quantitatively, the Encoder-Only model has a higher validation perplexity (approximately 21.5) and correspondingly higher losses, indicating that it struggles more with accurately predicting tokens. By contrast, the Full Transformer's best model achieves a much lower validation perplexity (around 5.8), reflecting significantly stronger predictive performance. This improvement is also evident in lower training and validation losses, demonstrating that the Full Transformer architecture converges more effectively. Qualitatively, the Encoder-Only approach often repeats words (e.g., "hat hat," "grass grass") and produces partial or incomplete phrases, suggesting limited ability to capture context fully. The Full Transformer's best model, on the other hand, yields more coherent translations, with fewer repeated tokens and more accurate vocabulary usage. It generally provides sentences that flow more naturally and retain key contextual elements from the source. Several factors contribute to these differences. First, the Full Transformer architecture includes both an encoder and a decoder, with the decoder trained specifically for autoregressive sequence generation. This design allows the model to attend to previously generated tokens as well as the encoder outputs, improving alignment and coherence. Second, the dual attention mechanisms (self-attention and cross-attention) in the decoder help it learn stronger dependencies across the entire sentence, thereby reducing repetition and enhancing word choice. Third, hyperparameter tuning (such as learning rate adjustments, number of heads, or feedforward dimension) further refines the Full Transformer's performance, lowering perplexity and loss. Finally, the Full Transformer's larger capacity and ability to focus on target-side context endow it with superior representational power for sequence-to-sequence tasks, resulting in higher-quality translations once adequately trained.

Compare Seq2Seq to Transformer (Best models)

Compare your Seq2Seq best model results to your Transformer best model results both quantitatively and qualitatively and explain the differences.

In comparing the best Seq2Seq model (LSTM with Attention) to the best Transformer model, we see substantial differences in both metrics and translation quality. Quantitatively, the Seq2Seq model's validation perplexity hovers around 23.8, whereas the Transformer's best configuration achieves a much lower perplexity of approximately 5.9, indicating it is more confident and accurate in predicting tokens. This gap is also evident in training losses and perplexities, where the Transformer converges to lower values overall. Qualitatively, the Seq2Seq model's outputs, while improved by attention, still exhibit occasional repetition of words or incomplete phrasing, suggesting some difficulty capturing long-range dependencies. In contrast, the Transformer's best model produces more fluent, contextually coherent translations with fewer repeated tokens. Its multi-head self-attention mechanisms, combined with a dedicated decoder, appear to handle complex sentence structures more effectively, resulting in more natural and context-aware predictions. The larger representational capacity of the Transformer architecture, along with well-tuned hyperparameters (such as learning rate and feedforward dimension), ultimately yields stronger performance across both training metrics and final translations.

Theory question

Derivation of $\frac{\partial L_2}{\partial U}$ for a 3-Time-Step RNN

We consider an RNN defined by the following equations:

$$a_t = Ux_t + Wh_{t-1} + b,$$

$$h_t = \tanh(a_t),$$

$$o_t = Vh_t + c,$$

$$\hat{y}_t = \text{softmax}(o_t),$$

$$L_t = \text{CE}(\hat{y}_t, y_t),$$

with the derivative of the tanh function given by:

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x),$$

and the hint that for the softmax-cross-entropy combination we have:

$$\frac{\partial L_t}{\partial o_t} = \hat{y}_t - y_t$$

Direct Contribution at $t = 2$

At time $t = 2$, we have:

$$a_2 = Ux_2 + Wh_1 + b,$$

$$h_2 = \tanh(a_2),$$

$$o_2 = Vh_2 + c,$$

and the loss L_2 is computed using $\hat{y}_2 = \text{softmax}(o_2)$.

Step 1. Compute the derivative with respect to o_2 :

$$\frac{\partial L_2}{\partial o_2} = \hat{y}_2 - y_2.$$

Step 2. Propagate through the output layer. Since $o_2 = Vh_2 + c$, we have:

$$\frac{\partial o_2}{\partial h_2} = V.$$

Thus, the derivative of L_2 with respect to h_2 is:

$$\frac{\partial L_2}{\partial h_2} = V^T(\hat{y}_2 - y_2).$$

Step 3. Backpropagate through the tanh nonlinearity. Since

$$h_2 = \tanh(a_2),$$

its derivative is:

$$\frac{\partial h_2}{\partial a_2} = 1 - \tanh^2(a_2).$$

Define:

$$\delta_2 \equiv \frac{\partial L_2}{\partial a_2} = (1 - \tanh^2(a_2)) \circ [V^T(\hat{y}_2 - y_2)],$$

where $\circ$ denotes the element-wise (Hadamard) product.

Step 4. Since

$$a_2 = Ux_2 + Wh_1 + b,$$

the derivative of a_2 with respect to U is:

$$\frac{\partial a_2}{\partial U} = x_2^T.$$

Thus, the **direct** contribution is:

$$\left(\frac{\partial L_2}{\partial U} \right)_{\text{direct}} = \delta_2 x_2^T.$$

Indirect Contribution via h_1

The loss L_2 also depends on U through the hidden state at $t = 1$. At $t = 1$, we have:

$$h_1 = \tanh(a_1) \quad \text{with} \quad a_1 = Ux_1 + Wh_0 + b.$$

Since h_1 influences a_2 through:

$$a_2 = Ux_2 + Wh_1 + b,$$

we first compute:

$$\frac{\partial a_2}{\partial h_1} = W.$$

Then, by the chain rule, the gradient of L_2 with respect to h_1 is:

$$\frac{\partial L_2}{\partial h_1} = \frac{\partial L_2}{\partial a_2} \frac{\partial a_2}{\partial h_1} = \delta_2 W.$$

Next, propagate this gradient through the tanh at $t = 1$. Since

$$h_1 = \tanh(a_1),$$

its derivative is:

$$\frac{\partial h_1}{\partial a_1} = 1 - \tanh^2(a_1).$$

Define:

$$\delta_1 \equiv \frac{\partial L_2}{\partial a_1} = (1 - \tanh^2(a_1)) \circ (W^T \delta_2).$$

Since

$$a_1 = Ux_1 + Wh_0 + b,$$

we have:

$$\frac{\partial a_1}{\partial U} = x_1^T.$$

Thus, the **indirect** contribution is:

$$\left(\frac{\partial L_2}{\partial U} \right)_{\text{indirect}} = \delta_1 x_1^T.$$

Summing the direct and indirect contributions, the total derivative is:

$$\boxed{\frac{\partial L_2}{\partial U} = \delta_2 x_2^T + \delta_1 x_1^T,}$$

where

$$\delta_2 = (1 - \tanh^2(a_2)) \circ [V^T(\hat{y}_2 - y_2)],$$

$$\delta_1 = (1 - \tanh^2(a_1)) \circ (W^T \delta_2).$$

Computation Graph Overview

The computation graph for the unrolled RNN (for three time steps) shows that the loss L_2 receives contributions from:

- **Direct Path:** $x_2 \rightarrow a_2 \rightarrow h_2 \rightarrow o_2 \rightarrow L_2$.
- **Indirect Path:** $x_1 \rightarrow a_1 \rightarrow h_1 \rightarrow a_2 \rightarrow h_2 \rightarrow o_2 \rightarrow L_2$.

Thus, the gradient $\frac{\partial L_2}{\partial U}$ is the sum of the gradients from these two paths.

Please see the **Computation Graph** to your right:

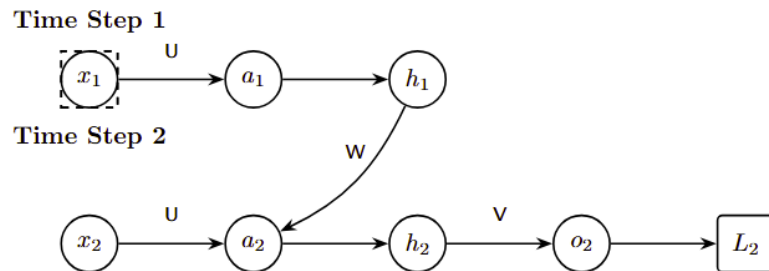


Figure 1: Unrolled RNN computation graph with parameter labels.

Paper discussion – Paper Choice 1

Short Paper Review (Paper 1):

This paper investigates whether Vision Transformers (ViTs) learn visual representations similarly to Convolutional Neural Networks (CNNs). The authors conduct an in-depth analysis using representation similarity metrics (such as Centered Kernel Alignment, or CKA) to compare the internal features of ViTs and CNNs. Their experiments reveal that, unlike CNNs whose lower and higher layers capture progressively abstracted local features, ViTs exhibit a more uniform representational structure across layers. This uniformity is attributed to the early aggregation of global information via self-attention and the strong influence of residual connections. A key strength of the paper lies in its thorough quantitative analysis across multiple models and datasets, offering new insights into how architectural choices (such as the absence of inherent spatial inductive biases in ViTs) affect feature learning. However, one potential weakness is that the study's conclusions are drawn from models pre-trained on specific large-scale datasets, which might limit the generality of the findings. My personal takeaway is that while ViTs depart from traditional CNN architectures by not hardcoding locality, they still manage to capture spatial information effectively through self-attention mechanisms. This suggests exciting possibilities for designing hybrid models that combine the strengths of both paradigms, particularly for tasks like object detection where spatial localization is crucial.

Paper Specific Questions (Paper 1):

Q1: Compare and contrast the learned features of ViTs and CNNs.

ViTs and CNNs differ fundamentally in how they learn representations due to their distinct architectural designs. CNNs use convolutional kernels that enforce locality, meaning that early layers typically focus on fine-grained, local features, while deeper layers build increasingly abstract representations. In contrast, ViTs rely on self-attention, which aggregates information globally from the very first layer. As a result, ViTs tend to have more uniform representations across layers, with lower and higher layers showing high similarity. For instance, Raghu et al. use Centered Kernel Alignment (CKA) to measure the similarity between representations across layers. Their experiments reveal that ViTs exhibit a grid-like pattern of uniform similarity, whereas CNNs (such as ResNets) display a stage-wise structure where early layers are distinctly different from later layers. This empirical evidence demonstrates that while CNNs build a hierarchical representation where they progressively abstract local features, ViTs maintain a consistent representational structure throughout the network. Moreover, the study shows that the strong influence of residual connections in ViTs facilitates the propagation of features from lower to higher layers, further contributing to their uniformity.

Q2: What is meant by spatial localization, and why might ViTs be advantageous for object detection?

Spatial localization refers to a network's ability to accurately preserve and pinpoint the spatial arrangement of features within an image. In CNNs, the use of local receptive fields inherently maintains spatial relationships, ensuring that features correspond to specific image regions. Although ViTs do not have these hardcoded local biases, they still preserve spatial information through their self-attention mechanisms and residual connections. Raghu et al. provides experimental evidence that lower layers of ViTs attend to both local and global information. Their analysis of average attention head distances shows that while early self-attention layers in ViTs capture local details, they also integrate global context. This is a balance that is crucial for tasks requiring precise localization. This dynamic ability to adjust the scope of attention allows ViTs to retain fine spatial details while understanding the overall scene, which is particularly advantageous for object detection. The paper's findings suggest that, even without the innate spatial inductive biases of CNNs, ViTs can achieve effective spatial localization through their flexible architecture, potentially offering better performance in scenarios where both global context and precise local feature mapping are essential.